C++ and objects copying

BIE-PA2 – Programming and Algorithmic II

Ladislav Vagner

Department of Theoretical Computer Science
Faculty of Information Technology
Czech Technical University in Prague
xvagner@fit.cvut.cz

March 9, 2026

Overview

- Copy constructor and copy assignment operator.
- Shallow and deep copy.
- Rvalue reference and `unique_ptr`.
- Move constructor and move assignment operator.
- Shallow copy and `shared_ptr`.
- Copy-on-write.
- Some array manipulation algorithms.

Class Array

Class Array as presented during the second lecture had copying disabled. What if it was not?

```
struct Array {
  Array() = default;
  ~Array() { delete[] data_; }

  // What if we do not disable copying?
  // Array(const Array&) = delete;
  // Array& operator = (const Array&) = delete;

  size_t size() const { return size_; }

  int& operator [] (size_t i) { return data_[i]; }
  int operator [] (size_t i) const { return data_[i]; }

  void push(int x);

  private:
  size_t size_ = 0, cap_ = 0;
  int *data_ = nullptr;
};
```

Class Array

```
void Array::push(int x) {
  if (size_ >= cap_) {
    cap_ = cap_ * 2 + 2;
    int *new_data = new int[cap_];
    for (size_t i = 0; i < size_; i++)
      new_data[i] = data_[i];
    delete[] data_;
    data_ = new_data;
  }

  data_[size_++] = x;
}

std::ostream& operator << (std::ostream& out,
                           const Array& a) {
  for (size_t i = 0; i < a.size(); i++) {
    if (i > 0) out << ", ";
    out << a[i];
  }
  return out;
}
```

Class Array

Let's attempt to copy the instance of class Array:

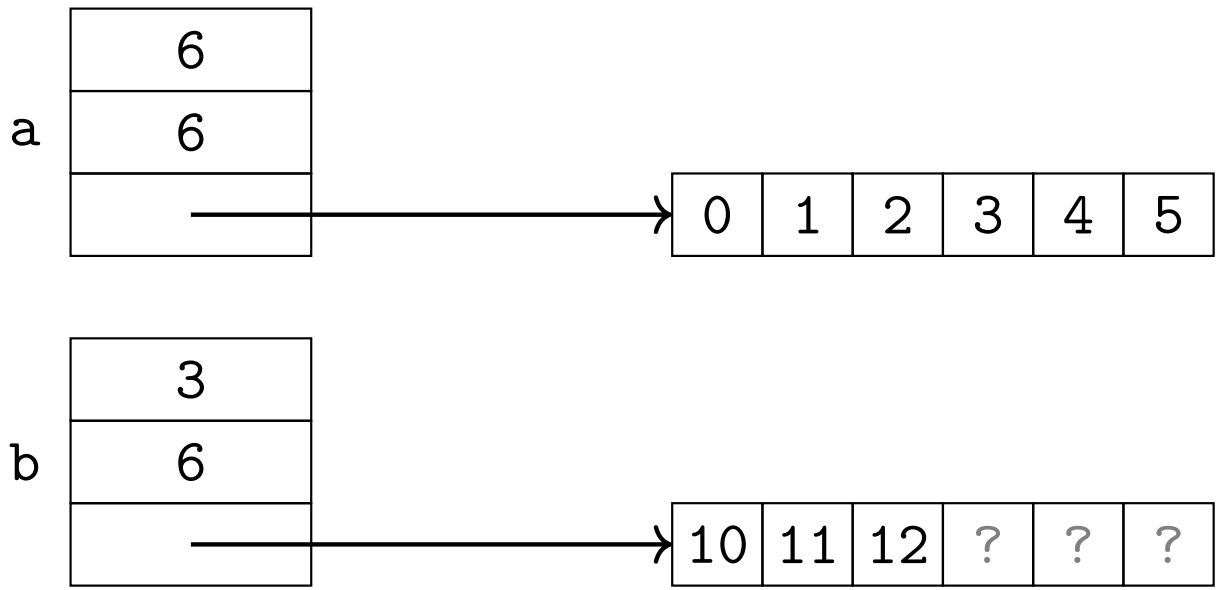
```
int main() {
  Array a, b;

  for (int i = 0; i < 6; i++) a.push(i);
  for (int i = 0; i < 3; i++) b.push(10 + i);
  std::cout << "a: " << a << std::endl; // 0 1 2 3 4 5
  std::cout << "b: " << b << std::endl; // 10 11 12

  b = a;
  b[1] = 99;

  std::cout << "a: " << a << std::endl; // 0 99 2 3 4 5
  std::cout << "b: " << b << std::endl; // 0 99 2 3 4 5

  return 0;
}
```

Why is a[1] also 99?

Class Array

Why was `a[1]` also set to 99?

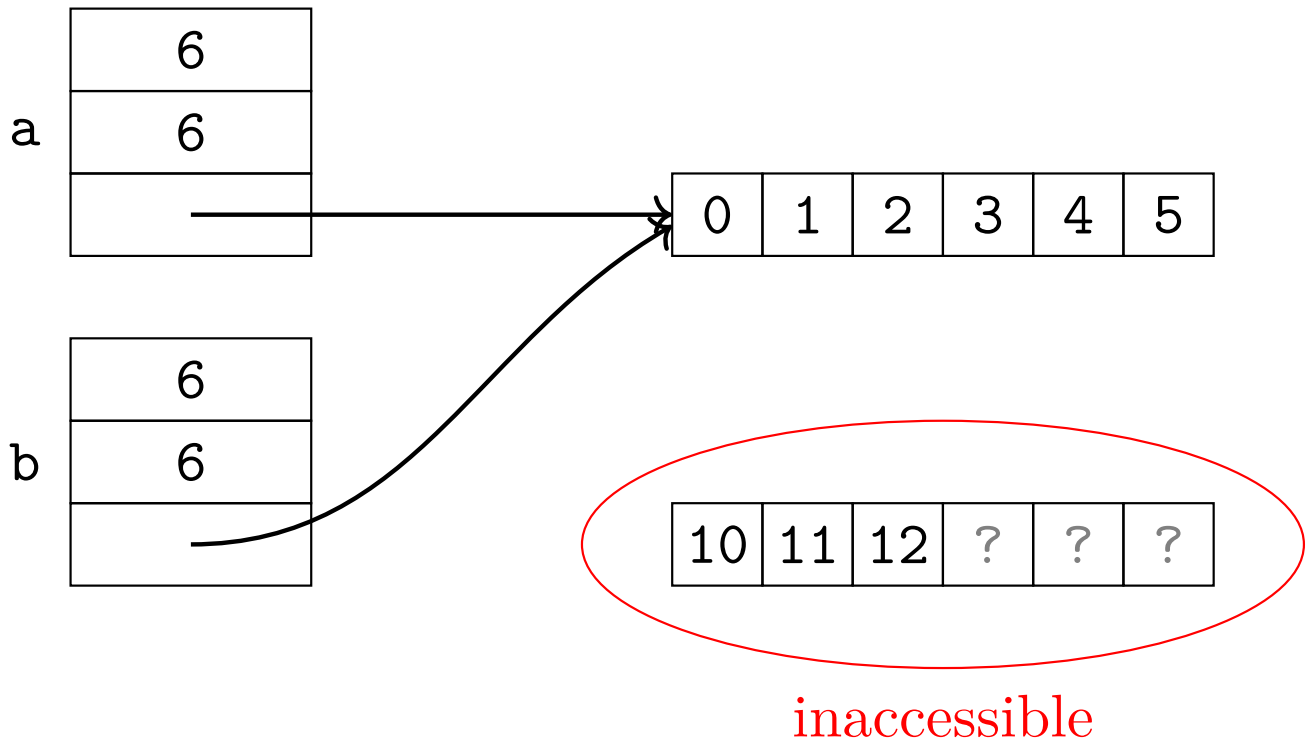
```
b = a;
b[1] = 99;
```



Class Array

Why was `a[1]` also set to `99`?

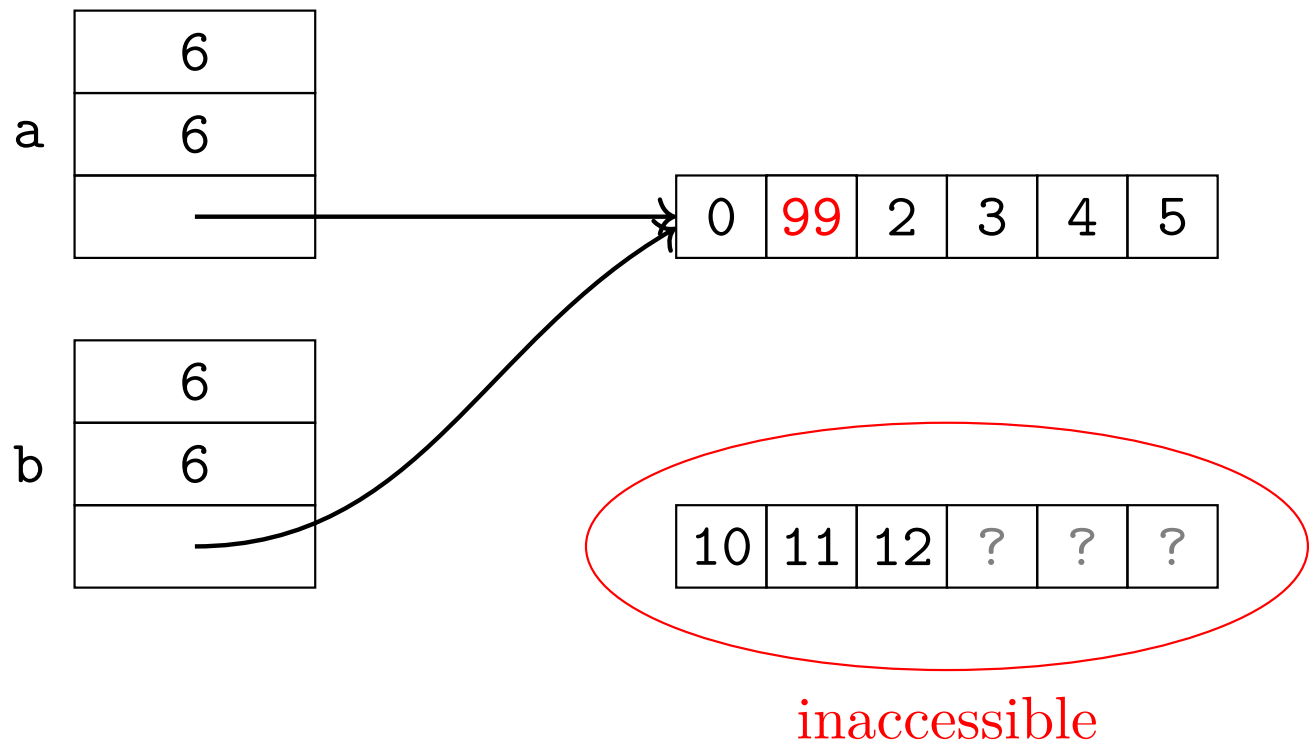
```
> b = a;
  b[1] = 99;
```



Class Array

Why was `a[1]` also set to 99?

```
b = a;
b[1] = 99;
```



Class `Array` – generated `operator =`

- If a class does not define a copy `operator =` or copy constructor, the compiler will generate one.
- The generated implementation calls copy `operator =` (or copy constructor) sequentially on each object's attribute.
  - Primitive types are copied bitwise (including pointers),
  - this leads to leaking memory, double freeing for pointers to data owned by the object,
  - often also to undesired memory sharing.
- The intent was to create a standalone copy:
  - the memory block needs to be copied,
  - the memory block we loose access to must be freed.

Class Array – implementation of copying

```
struct Array {
  Array(const Array&);
  Array& operator = (const Array&);
  ...
};

Array::Array(const Array& o) : size_(o.size_),
    cap_(o.cap_), data_(cap_ ? new int[cap_] : nullptr) {
  for (size_t i = 0; i < size_; i++) data_[i] = o.data_[i];
}

Array& Array::operator = (const Array& o) {
  if (&o == this) return *this;

  delete[] data_;
  size_ = o.size_;
  cap_ = o.cap_;
  data_ = cap_ ? new int[cap_] : nullptr;
  for (size_t i = 0; i < size_; i++) data_[i] = o.data_[i];

  return *this;
}
```

Class `Array` – implementation of copying

- Why does the `operator =` implementation contain:

  ```
  if (&o == this) return *this;
  ```

- Why doesn't the copy constructor contain this `if`?
- What is the difference between `&o == this` and `o == *this`?
- **Rule of three**: If the class user defines
  - copy constructor,
  - copy `operator =`, or
  - destructor,

  it should implement all three.
- **Note**: Statement `Array c = a` calls a copy constructor, **not** the copy `operator =`.

Copying more generally

- *Deep copy*:
  - A copy that is completely standalone from the original object,
  - typically the referred objects need to be copied too,
  - it may be time and space consuming,
  - Can be implemented more efficiently using copy-on-write technique.
- *Shallow copy*:
  - Copies only the object itself, references to other objects are shared,
  - typically very fast operation,
  - the copies are not standalone,
  - In languages without garbage collector it may be needed to handle memory sharing for shared objects.
- A copy of pointers in C was shallow,
- in C++ we expect the copying to implement a deep copy,
- objects using pointers need to be handled specially.

Implementation of copying

- A copy constructor and copy `operator =` were almost identical.
- Can one implement the other?

Implementation of copying

- A copy constructor and copy `operator =` were almost identical.
- Can one implement the other?
- Yes, using copy-and-swap technique:

```
Array& Array::operator = (Array o) { // by value
  using std::swap;
  swap(size_, o.size_);
  swap(cap_, o.cap_);
  swap(data_, o.data_);
  return *this;
}
```

- Notice the `using std::swap`:
  - The function `swap` swaps its arguments,
  - there is a generic implementation `std::swap`,
  - classes may provide a specialized (and thus more efficient) overloads,
  - these should be in the namespace of the given class,
  - thus we need to write just `swap`, not `std::swap`, but also to have the implementation from `std` available.

Implementation of copying

- A better copy-and-swap implementation is:

```
void swap(Array& a, Array& b) {
  using std::swap;
  swap(a.size_, b.size_);
  swap(a.cap_, b.cap_);
  swap(a.data_, b.data_);
}

Array& Array::operator = (Array o) { // by value
  swap(*this, o); return *this;
}
```

- The function `swap` must typically be a `friend`.
- Note that the implementation

```
Array& operator = (Array o) {
  std::swap(*this, o); return *this;
}
```

leads to an infinite recursion since `std::swap` calls `operator =`.

Implementation of copying and moving

- Implementation using copy-and-swap does not allow a reuse of allocated memory (which is not an issue, typically),
- It also handles assignment `a = a` inefficiently.
- What if we know the source object will no longer be needed after copying?

Implementation of copying and moving

- Implementation using copy-and-swap does not allow a reuse of allocated memory (which is not an issue, typically),
- It also handles assignment `a = a` inefficiently.
- What if we know the source object will no longer be needed after copying?
- We would like to move it instead of copying,
- we need to find a way to denote an object whose content will not be needed anymore and thus can be moved.

Implementation of copying and moving

- Implementation using copy-and-swap does not allow a reuse of allocated memory (which is not an issue, typically),
- It also handles assignment `a = a` inefficiently.
- What if we know the source object will no longer be needed after copying?
- We would like to move it instead of copying,
- we need to find a way to denote an object whose content will not be needed anymore and thus can be moved.
- Since C++11 these are rvalue references.

Rvalue references

- A simplified terminology (originally from C):
  - *lvalue* – can be on the left hand side of =. I.e. it is a value that is not temporary. Typically variables and (lvalue) reference.
  - *rvalue* – a temporary value. Return value of functions which is not a lvalue reference.
- It is denoted by && instead of & as in the common (lvalue) references.
- Non-`const` lvalue references are never bound to rvalue:

```
void foo(int&);
foo(5); // Error
```

- Rvalue references are only bound to rvalues and have priority over `const` lvalue references:

```
void foo(const int&);
void foo(int&&);
foo(5); // Calls foo(int&&)
```

- **Note**: A variable of rvalue reference type is an lvalue!

Rvalue references

- **Note**: A variable of rvalue reference type is an lvalue!

```
void bar(int&&);
void foo(int&& i) {
  bar(i); // Error, i is an lvalue
}
```

- The variable `i` must be made an rvalue:

```
void foo(int&& i) {
  bar(static_cast<int&&>(i)); // OK
}
```

- This operation is very common, therefore there is `std::move`, which does this for any type:

```
void foo(int&& i) {
  bar(std::move(i)); // OK
}
```

- Function `std::move` does not move anything, just marks the object as moveable.

std::move and std::swap

- Both are function templates,
- function std::move is primarily only static_cast to an rvalue reference, but its implementation requires other techniques (universal reference).
- Simplified implementation of std::swap:

```
namespace std {

template < typename T >
void swap(T& a, T& b) {
  T tmp(std::move(a));
  a = std::move(b);
  b = std::move(tmp);
}

}
```

Move implementation

A class can have a move constructor and move `operator =`:

```
struct Array {
  Array(Array&& o)
    : size_(o.size_), cap_(o.cap_), data_(o.data_) {
    o.size_ = o.cap_ = 0; // Cleanup o
    o.data_ = nullptr;
  }
  Array& operator = (Array&& o) {
    if (&o == this) return *this;

    delete[] data_;
    size_ = o.size_;
    cap_ = o.cap_;
    data_ = o.data_;

    o.size_ = o.cap_ = 0; // Cleanup o
    o.data_ = nullptr;
    return *this;
  }
  ...
};
```

Move implementation

- With copy-and-swap, the move `operator =` is not needed,
- the operator receives a value, which is on the call side either created by a copying, or moving.
- If the object initialization is cheap, we can use `swap` to also implement the move constructor.

```
struct Array {
  Array(Array&& o) : Array() { swap(*this, o); }

  friend void swap(Array& a, Array& b) {
    using std::swap;
    swap(a.size_, b.size_);
    swap(a.cap_, b.cap_);
    swap(a.data_, b.data_);
  }

  Array& operator = (Array o) { // by value
    swap(*this, o); return *this;
  }
  ...
};
```

Copying and moving – summary

- **The rule of five**: the rule of three plus moving.
- After moving, the source object must be in a valid state:
  - only methods without preconditions can be called,
  - typically these are a destructor and `operator =`.
- If there is no move constructor (move `operator =`), the copying one is used,
- the compiler generates copy constructor and `operator =`, always if it is not defined,
- the compiler generates the move constructor and `operator =`, if none of them is declared and neither is their copy variant, nor destructor.
- **The rule of zero**: better leave all of copy/move constructor/`operator =` and destructor on the compiler if possible.

A smart pointer – `std::unique_ptr`

- A class representing an exclusive ownership of a dynamically allocated object can be implemented using moving semantics,
- the objects of this class can't be copied, just moved.
- If the class is destroyed it also frees the owned object.
- Thanks to operator overloading it may have the same interface as a pointer.
- Implemented using class `std::unique_ptr`.

```
int main() {
  // Cannot use = because the constructor is explicit
  std::unique_ptr<int> i(new int);
  // Array
  std::unique_ptr<float[]> arr1(new float[15]);
  // Or make_unique, arguments are passed to constructor
  auto j = std::make_unique<int>(3);
  // make_unique also works with arrays
  auto arr2 = std::make_unique<double[]>(38);
} // All memory released
```

Use of move and unique_ptr it class Array

```
struct Array {
  using T = std::string;

  Array() = default;
  Array(const Array&);
  Array(Array&& o);
  Array& operator = (Array o);
  ~Array() = default; // Explicitly defaulted

  friend void swap(Array& a, Array& b);

  size_t size() const { return size_; }
  T& operator [] (size_t i) { return at_(i); }
  const T& operator [] (size_t i) const { return at_(i); }

  void push(T x); // by value

  private:
  T& at_(size_t i) const; // !!
  size_t size_ = 0, cap_ = 0;
  std::unique_ptr<T[]> data_;
};
```

Use of move and unique_ptr it class Array

```
Array::Array(const Array& o) : size_(o.size_), cap_(o.cap_),
    data_(cap_ ? std::make_unique<T[]>(cap_) : nullptr) {
  for (size_t i = 0; i < size_; i++)
    data_[i] = o.data_[i];
}

Array::Array(Array&& o) {
  swap(*this, o);
}

Array& Array::operator = (Array o) {
  swap(*this, o); return *this;
}

void swap(Array& a, Array& b) {
  using std::swap;
  swap(a.size_, b.size_);
  swap(a.cap_, b.cap_);
  swap(a.data_, b.data_);
}
```

Use of move and unique_ptr it class Array

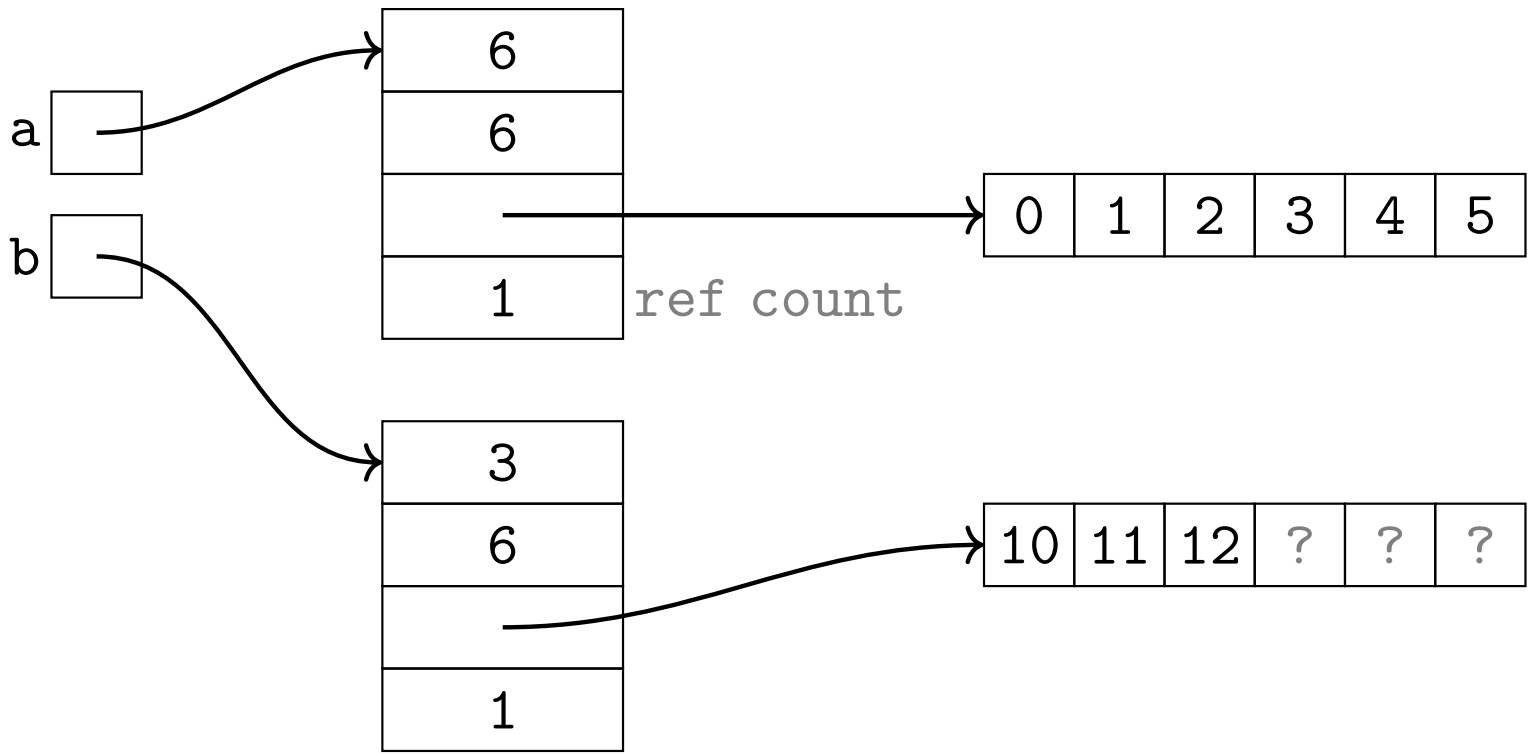
```
Array::T& Array::at_(size_t i) const {
  if (i >= size_) throw std::out_of_range("");
  return data_[i];
}

void Array::push(T x) {
  if (size_ >= cap_) {
    cap_ = cap_ * 2 + 2;
    auto new_data = std::make_unique<T[]>(cap_);

    for (size_t i = 0; i < size_; i++)
      new_data[i] = std::move(data_[i]);

    using std::swap;
    swap(data_, new_data);
  } // Old data_ destroyed

  data_[size_++] = std::move(x);
}
```

Shallow copy and counting references

- The issue with shallow copy was too an early (and repeated) freeing of memory.
- The problems with shared resources may be solved if counted references (and copy-on-write) techniques are used.
- Object `obj` maintains a counter which keeps track on the number of other objects referencing it:
  - If a new object referencing `obj` is created, the counter in `obj` is incremented,
  - if an object referencing `obj` is destroyed, the counter in `obj` is decremented,
  - if the counter reaches 0, the object `obj` is released.
- The object `obj` will only exist for as long as there is a reference to it.
- Cyclic references lead to memory leaks (can be solved using class `std::weak_ptr`).

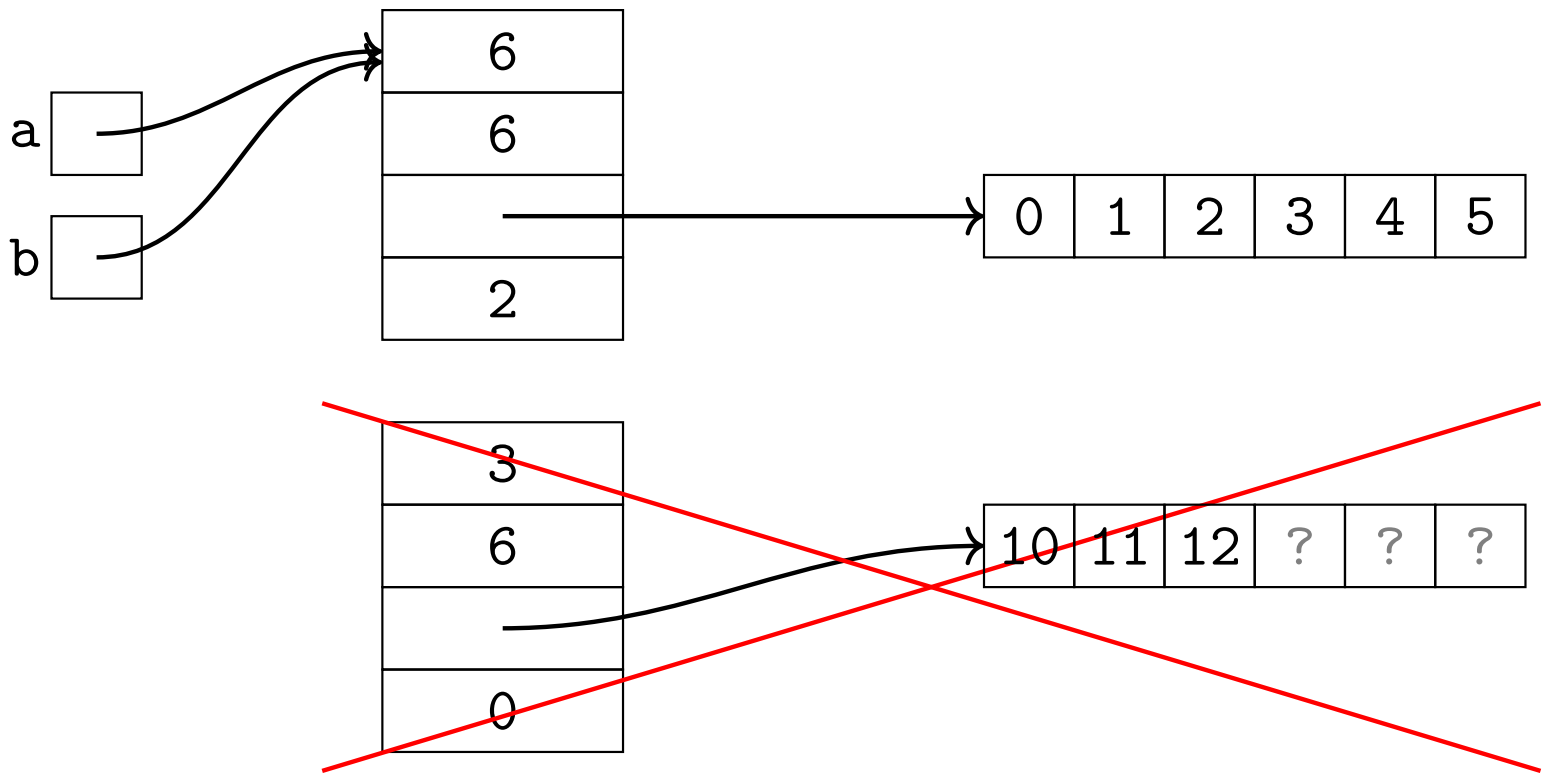
Class SharedArray

```
SharedArray a, b;
... // Fill a and b

b = a;
```



Class SharedArray

```
SharedArray a, b;
... // Fill a and b

b = a;
```



Class SharedArray

```
struct SharedArray {
  SharedArray();
  SharedArray(const SharedArray&);
  // SharedArray(SharedArray&& o); no moving
  SharedArray& operator = (SharedArray o);
  ~SharedArray();
  friend void swap(SharedArray& a, SharedArray& b);

  size_t size() const;
  int& operator [] (size_t i);
  int operator [] (size_t i) const;
  void push(int);
  friend std::ostream& operator << (std::ostream& out,
                                    const SharedArray& a);
  private:
  struct Impl {
    Array arr;
    size_t ref_count = 1;
  };
  Impl *impl_;
};
```

Class SharedArray

- Default constructor allocates a shared array.
- Move constructor would need to allocate after moving, thus it does not make sense to implement it.

```
SharedArray::SharedArray() : impl_(new Impl) {}

SharedArray::SharedArray(const SharedArray& o)
  : impl_(o.impl_) {
  impl_->ref_count++;
}

SharedArray& SharedArray::operator = (SharedArray o) {
  swap(*this, o);
  return *this;
}

SharedArray::~SharedArray() {
  if (--impl_->ref_count == 0) delete impl_;
}
```

Class SharedArray

The heavy lifting is delegated to the referenced Array:

```
void swap(SharedArray& a, SharedArray& b) {
  using std::swap; swap(a.impl_, b.impl_);
}

size_t SharedArray::size() const { return impl_->arr.size(); }

int& SharedArray::operator [] (size_t i) {
  return impl_->arr[i];
}
int SharedArray::operator [] (size_t i) const {
  return impl_->arr[i]; // We invoke non-const op[]
}

void SharedArray::push(int x) { impl_->arr.push(x); }

std::ostream& operator << (std::ostream& out,
                           const SharedArray& a) {
  return out << a.impl_->arr;
}
```

Class `SharedArray` – summary (and `shared_ptr`)

- Implementation builds on an added reference counter to the class `Array` and method delegation.
- Adding a counter is a general technique, anytime we need to leverage shallow copying.
- For that reason, the standard library implements `std::shared_ptr`.
- The `shared_ptr` has a move-constructor, which sets the source's referred object to `nullptr`.
- Which was not desired in the implementation of `SharedArray`.

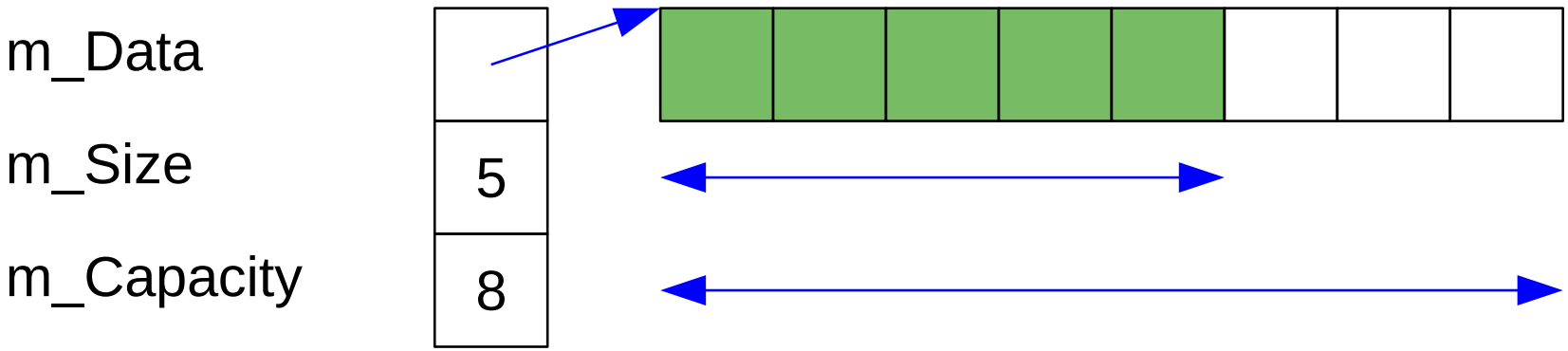
Copy-on-write

- If the copied object are not modified, a shallow copy implemented using reference counting and deep copy can't be distinguished.
- Shallow copying can be used until a modification is requested, at which time a deep copy must be created.
- We thus have a fast, and with not that many modifications also memory efficient, copying, which to the user seems to implement deep copying.
- To save more memory, the object can be partitioned into smaller pieces on which the copy-on-write is implemented individually.
- The drawback is a higher overhead and a necessity to keep track of all ways the object is accessed.
- More details at the proseminar.

Some array manipulating algorithms

- addition of an element,
- deletion of an element,
- filtering,
- partition,
- array rotation.

Addition/deletion of a last element



- addition is trivial operation (we have to ensure there is enough capacity),
- deletion only decreases the number of elements in the array,
- complexity $O(1)$ (amortized),
- in STL typically available as `push_back` and `pop_back`.

Addition/deletion of a last element

```
TDATA  * m_Data;
size_t   m_Size, m_Capacity;
...
void push_back ( const TDATA & newElement ) {
  if ( m_Size == m_Capacity )
  {
     m_Capacity = m_Capacity * GROW + ADD;
     TDATA * tmp = new TDATA [ m_Capacity ];
     for  ( size_t i = 0; i < m_Size; i ++ )
       tmp[i] = m_Data[i];  // !!!
     delete [] m_Data;
     m_Data = tmp;
  }
  m_Data[m_Size++] = newElement;
}
```

Addition/deletion of a last element

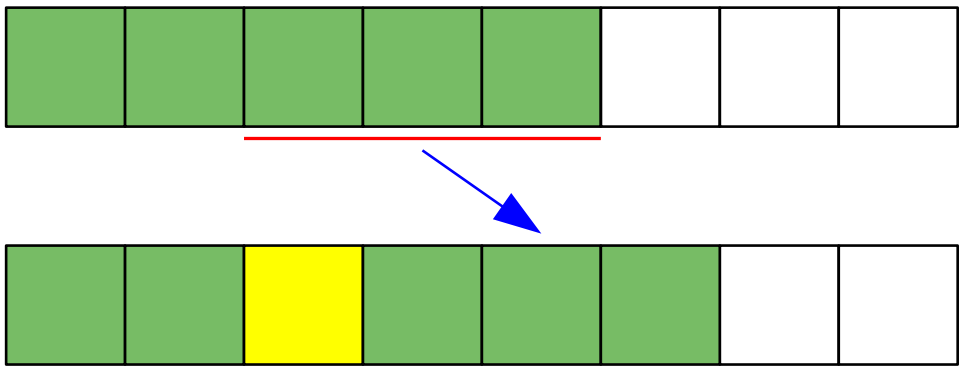
```
TDATA  * m_Data;
size_t   m_Size, m_Capacity;
...
void pop_back ( void ) {
  if ( m_Size > 0 )
    m_Size --;
}
```

Addition/deletion of a middle element

m_Size = 5

m_Capacity = 8

insPos = 2



- addition must shift elements at higher indexes to the right,
- deletion must shift elements at higher indexes to the left,
- complexity $O(n)$,
- in STL typically available as `insert` and `erase`.

Addition/deletion of a middle element

```
TDATA  * m_Data;
size_t   m_Size, m_Capacity;
...
void insert ( size_t insPos, const TDATA & newElement ) {
  // insPos is an iterator in STL
  if ( m_Size == m_Capacity )
    ... // resize

  for ( size_t i = m_Size; i > insPos; i -- )
    m_Data[i] = m_Data[i-1];

  m_Data[insPos] = newElement;
  m_Size ++;
}
```

Addition/deletion of a middle element

```
TDATA  * m_Data;
size_t   m_Size, m_Capacity;
...
void erase ( size_t delPos ) {
  // delPos is an iterator in STL

  if ( delPos >= m_Size )
    return;

  m_Size --;

  for ( size_t i = delPos; i < m_Size; i ++ )
    m_Data[i] = m_Data[i+1];
}
```

Addition/deletion of a middle element

If the order of elements is not important, constant complexity is achievable:

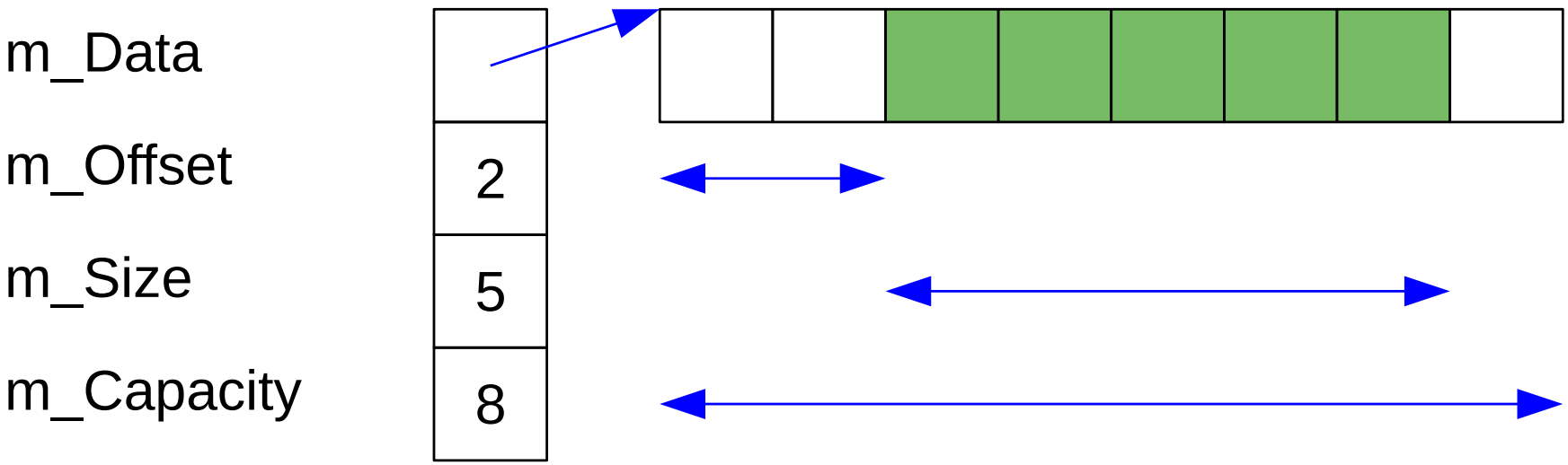
```
TDATA  * m_Data;
size_t   m_Size, m_Capacity;
...
void erase_fast ( size_t delPos ) {
  // this method is not included in STL

  if ( delPos >= m_Size )
    return;

  m_Data[ delPos ] = m_Data[ --m_Size ];
}
```

Addition/deletion of a first element

- For the first element, the algorithm for middle may be used (middle element, position=0),
- complexity $O(n)$,
- there is no better approach for a plain array and `std::vector`,
- however, a queue (called `std::deque` in STL) is able to do this in $O(1)$.

Fast addition/deletion of a first element

A possible implementation:



- the array is placed to the middle of the allocated space,
- the offset of the first element is stored,
- the implementation of `std::deque` is different and more complicated.

Array content filtering

- The array contains some elements,
- we are to remove all occurrences of `x`,
- linear complexity is required,
- we don't want to allocate memory (another array),
- STL functions `std::remove` and `std::remove_if` "shake" the wanted array content to the beginning, will not erase the rest (may be achieved by combining with `erase`).

Array content filtering

```
void remove ( TDATA * data, size_t & n,
              const TDATA & x ) {
  // std::remove has a generalized
  // interface with iterators

  size_t newSize = 0;
  for ( size_t i = 0; i < n; i ++ )
    if ( data[i] != x )
      data[newSize ++] = data[i];
  n = newSize; // std::remove does not resize
}
```

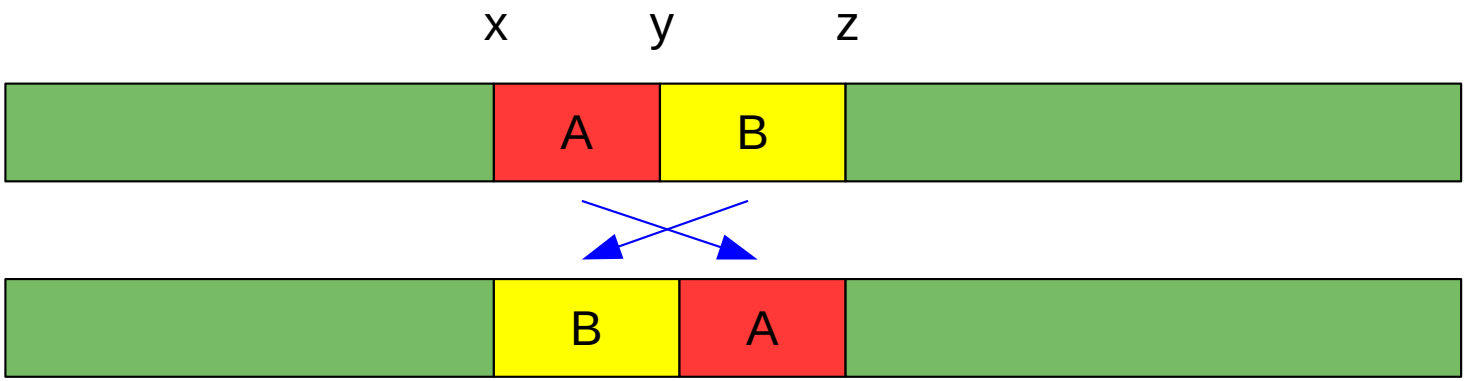
Array content partition

- The array contains some elements,
- array elements that satisfy a given property shall be placed at the beginning,
- array elements that don't satisfy the property shall be placed at the end,
- linear complexity is required,
- we don't want to allocate memory (another array),
- used in algorithm quick sort,
- STL function `std::partition`.

Array content partition

```
size_t partition ( TDATA * data, size_t n,
                   const TDATA & pivot ) {
  // std::partition has a generalized
  // interface with iterators and a predicate
  size_t st = 0, en = n;
  while ( st < en )  {
     while ( st < en && data[st] < pivot )
       st ++;
     while ( st < en && data[en - 1] >= pivot )
       en --;
     if ( st < en )
       swap ( data[st++], data[--en] );
  }
  return st;
}
```

Rotation inside an array



- We want to swap two continuous array ranges (spans),
- similar to a block operation in text editor (cut & paste),
- linear complexity is required,
- we don't want to allocate memory (another array),
- STL function `std::rotate`.

Rotation inside an array

Naive solution:

- `y-x` times shift all elements to the left by 1, up to the quadratic complexity,
- create a temporary array of `y-x` size, linear complexity, requires additional memory.

Efficient solution:

Rotation inside an array

Naive solution:

- `y-x` times shift all elements to the left by 1, up to the quadratic complexity,
- create a temporary array of `y-x` size, linear complexity, requires additional memory.

Efficient solution:

- shift elements by `y-x` positions to the left, jump with `y-x` steps (modulo `z-x`),

Rotation inside an array

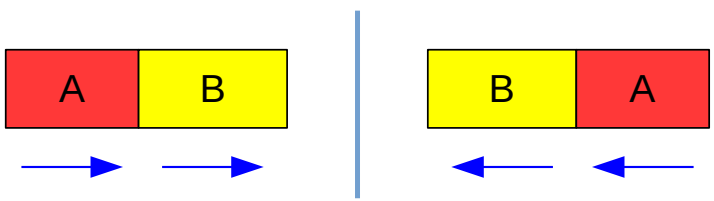
Naive solution:

- `y-x` times shift all elements to the left by 1, up to the quadratic complexity,
- create a temporary array of `y-x` size, linear complexity, requires additional memory.

Efficient solution:

- shift elements by `y-x` positions to the left, jump with `y-x` steps (modulo `z-x`),
- use a trick with reverse:

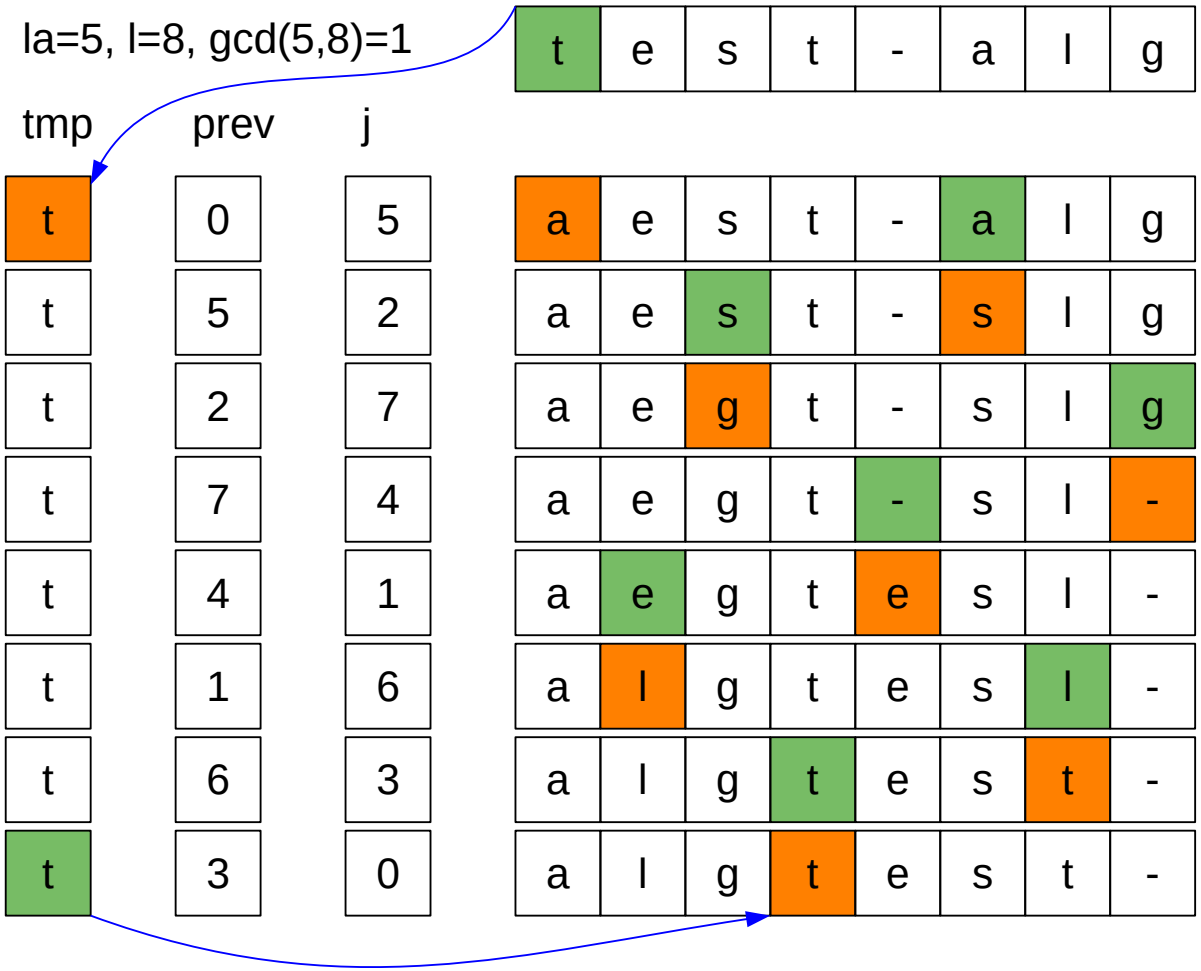
$$(AB)^R = B^R A^R$$



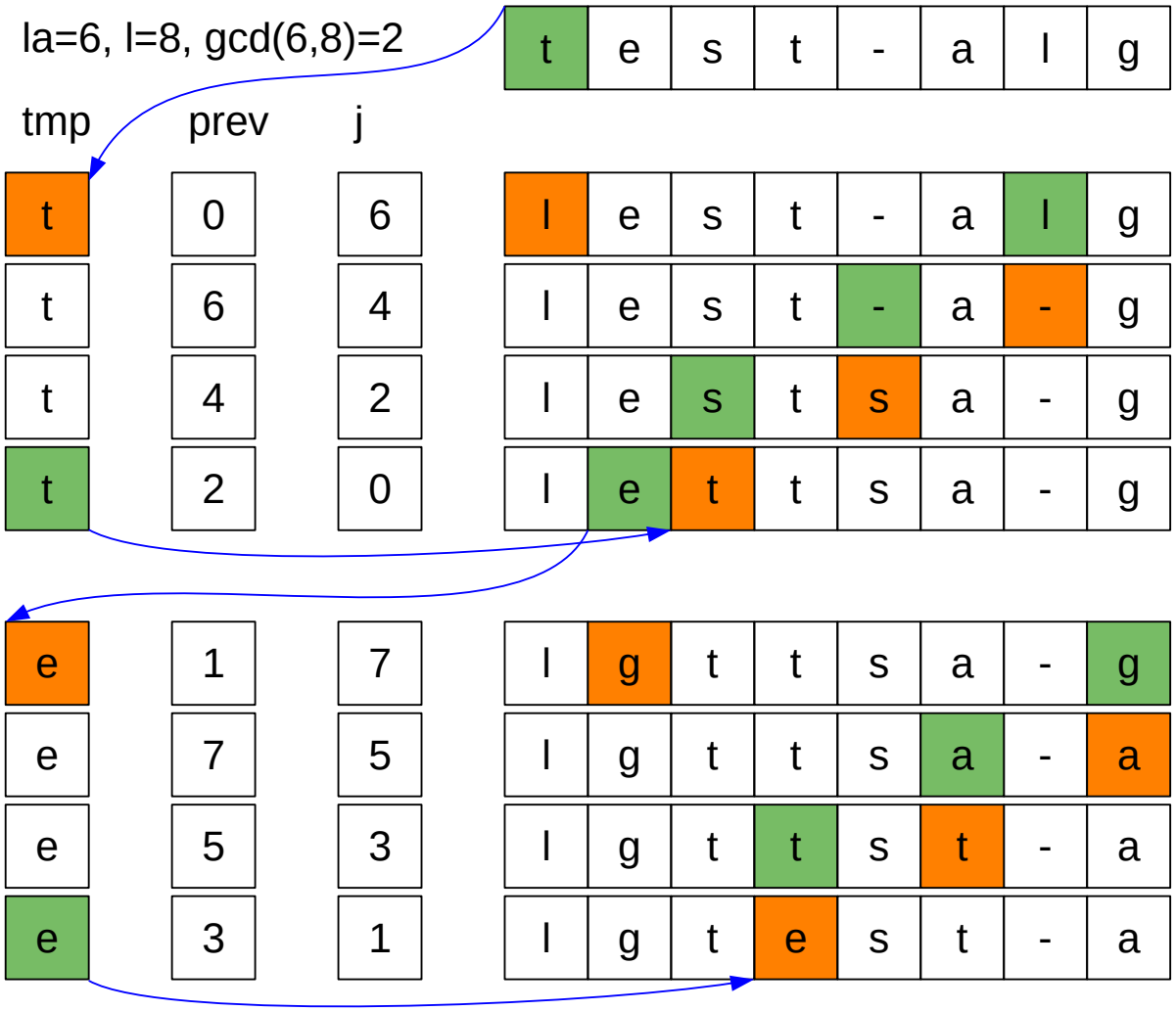
Rotation inside an array

```
void rotate ( TDATA * data, size_t x,
              size_t y, size_t z ) {
  size_t la = y - x, l = z - x, iter = gcd ( la, l );
  data += x;
  for ( size_t i = 0; i < iter; i ++ ) {
    TDATA tmp = data[i];
    size_t prev = i;
    for ( size_t j = i + la; j != i;  ) {
      data[prev] = data[j];
      prev = j;
      j = ( j + la ) % l;
    }
    data[prev] = tmp;
  }
}
```

Rotation inside an array



Rotation inside an array

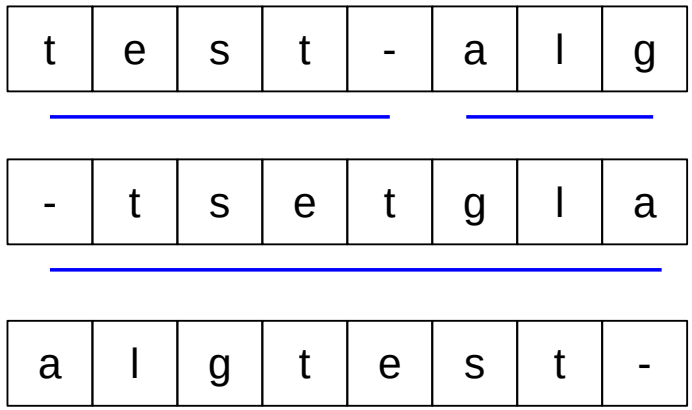


Rotation inside an array

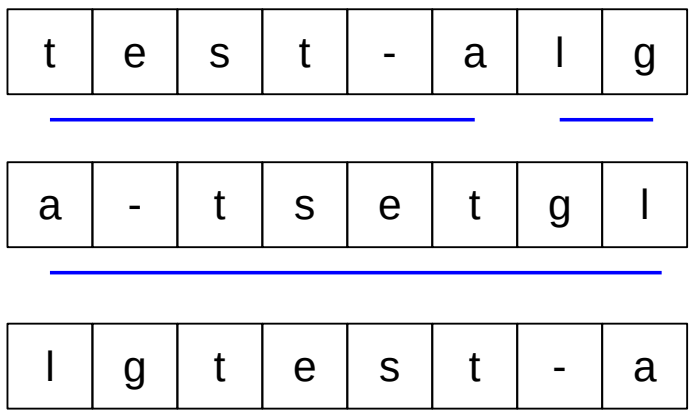
```
void reverse ( TDATA * st, TDATA * en ) {
  while ( st < en )
    swap ( *st++, *--en );
}
void rotate ( TDATA * data, size_t x,
              size_t y, size_t z ) {
  reverse ( data + x, data + y );
  reverse ( data + y, data + z );
  reverse ( data + x, data + z );
}
```

Rotation inside an array

la=5, l=8



la=6, l=8



Questions and answers

Questions ...